



SECURITY ASSESSMENT

Provided by Accretion Labs Pte Ltd. for Hylo
December 08, 2025
A25HYL1



AUDITORS

Role	Name
Lead Auditor	Robert Reith (robert@accretion.xyz)
Auditor	Mahdi Rostami (mahdi@accretion.xyz)
Auditor	Niklas Brymko (niklas@accretion.xyz)

CLIENT

Hylo (<https://hylo.so>) is a protocol implementing a dual-token decentralized stablecoin system. The hyUSD stablecoin is backed by liquid staking tokens, while xSOL is a leveraged Sol derivative which absorbs all price movements. In addition, the protocol implements a stability pool which allows stakers to profit from Solana's liquid staking yield, while providing a mechanism to stabilize the protocol. Hylo engaged Accretion to conduct an assessment of the exchange and the stability pool.

ENGAGEMENT TIMELINE



AUDITED CODE

Program 1

ProgramID: HYEXCHtHkBagdStcJcP3xbbb9B7sdMdWxFNj6mdsG4hn

Repository: <https://github.com/hylo-so/protocol>

Program 2

ProgramID: HysTabVUfmQBfcmzu1ctRd1Y1fxd66RBpboy1bmtDSQQ

Repository: <https://github.com/hylo-so/protocol>

ASSESSMENT

The security assessment of Hylo's stablecoin and leveraged Sol protocol revealed a well-designed system with one high-severity and several medium-severity issues that were successfully addressed. The audit uncovered 16 findings primarily focused on price feed integrity, fee calculations, and yield accounting mechanisms. All immediately relevant findings were fully remediated during the audit period.

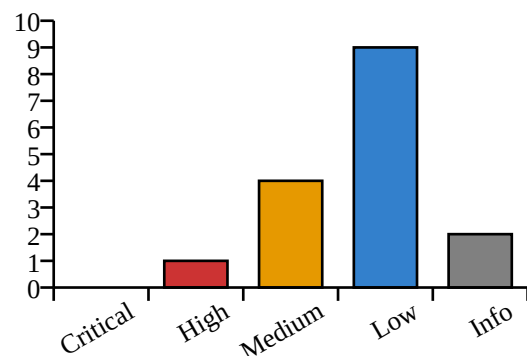
CODE ASSESSMENT

The Anchor codebase implemented a sophisticated dual-token stablecoin system with stability pool mechanics and LST integration. The architecture demonstrated strong design principles with clear separation between minting, swapping, and yield operations. Test coverage adequately addressed core functionality, though edge cases around yield timing and price feed staleness required additional attention.

KEY FINDINGS

Our top finding was a high-severity issue allowing price manipulation through non-canonical price feeds. Medium-severity concerns included fee calculations using incorrect stability modes, missing slippage protection in swap operations, and yield miscalculation when deposits occur before harvesting. Low-severity findings addressed admin controls and operational optimizations.

SEVERITY DISTRIBUTION



ENGAGEMENT SCOPE

The scope of this security assessment was a full review of the following items:

Item 1: Hylo Exchange

Link: <https://github.com/hylo-so/protocol>

Commit: da92204a20e172f1ffb92798552a183040d11947

Program ID: HYEXCHtHkBagdStcJCp3xbbb9B7sdMdWXFNj6mdsG4hn

Audit Result:

- **Audited Commit:** 0c3c6d34c5c50d94425f150f86913e2be7431cb7
- **Build Hash:** 7d4e5ff843530aeefb44e3356edec81999ee0f71bf0516b6c9ab16cb577d0fa
- **Status:** Verified
- **Comment:** Verified build matches on-chain

Item 2: Hylo Stability Pool

Link: <https://github.com/hylo-so/protocol>

Commit: da92204a20e172f1ffb92798552a183040d11947

Program ID: HysTabVUfmQBfcmzu1ctRd1Y1fxd66RBpboy1bmtDSQQ

Audit Result:

- **Audited Commit:** 0c3c6d34c5c50d94425f150f86913e2be7431cb7
- **Build Hash:** b784e8ba0c59b014cad4b66ab4ade89e09b2eb0630874530f44dd87d496ff63d
- **Status:** Verified
- **Comment:** Verified build matches on-chain

ISSUES SUMMARY

ID	TITLE	SEVERITY	STATUS
ACC-H1	Non-Canonical Price Feeds Allow Historical Price Manipulation	HIGH	FIXED
ACC-M1	Fee Calculation Uses Targeted Mode Instead of Current Mode	MEDIUM	FIXED
ACC-M2	Swap Instruction Lacks Slippage Protection	MEDIUM	FIXED
ACC-M3	Yield Miscalculation Due to Deposits/Withdrawals Before Harvest	MEDIUM	FIXED
ACC-M4	LST Yield Loss Due to `prev_price_sol` Updates	MEDIUM	WONTFIX
ACC-L1	Admin can reinitialize lst registry	LOW	FIXED
ACC-L2	Incorrect `projected_stability_mode` Calculation During Mint and Swap	LOW	WONTFIX
ACC-L3	First Share Manipulation Risk in Stability Pool	LOW	WONTFIX
ACC-L4	Unnecessary Admin Signature Requirement in Stability Pool `UpdateAdmin`	LOW	FIXED
ACC-L5	Levercoin NAV Incorrectly Non-Zero in Depeg Mode	LOW	FIXED
ACC-L6	Unnecessary Enforcement of ATA for Users	LOW	FIXED
ACC-L7	Incorrect Validation of Harvestable Stablecoin Amount in `yield_harvest`	LOW	FIXED
ACC-L8	Collateral Ratio Skew from Delayed `harvest_yield`	LOW	FIXED
ACC-L9	Setting Arbitrarily high fees can lead to DoS	LOW	FIXED
ACC-I1	LST portfolio rebalancing	INFO	WIP
ACC-I2	Incorrect Rebalance Behavior in Stability Pool	INFO	FIXED

DETAILED ISSUES

ACC-H1 Non-Canonical Price Feeds Allow Historical Price Manipulation

HIGH

FIXED

Description

We found that the protocol does **not enforce canonical accounts** for `sol_usd_pyth_feed`. Any account with the same **feed ID** can be submitted, allowing anyone to create a `PriceUpdateV2` account and publish historical prices.

For example, with a 5-minute age window:

- Transaction 1 uses price `t2` (latest).
- Transaction 2 uses price `t1` (older, $t1 < t2$) from a custom account.

While the protocol correctly validates publish and update times, this issue allows users to submit historical valid prices within the allowed time window (which was configured to be 60s), potentially manipulating outcomes.

Location

https://github.com/hylo-so/protocol/blob/2381c3602dae7bc0be1e2576d2824dd69fab2ba9/programs/hylo-exchange/src/instructions/mint_levercoin.rs#L106-L109

Relevant Code

```
#[account(
    constraint = sol_usd_pyth_feed.price_message.feed_id == SOL_USD,
)]
pub sol_usd_pyth_feed: Account<'info, PriceUpdateV2>,
```

Mitigation Suggestion

- Use a **canonical feed account** managed and sponsored by Pyth.
- Alternatively, use a **personal, persistent account** that you control and update yourself.

Remediation

Fixed in commit `38e0c79d00de5b67427cc8d9fe64cdd12e98f60c`.

Description

The protocol's fee mechanism currently works as follows:

1. Compute the **projected stability mode** after the transaction.
2. Apply the fee rate corresponding to that mode.

This approach works correctly in some scenarios:

- Mode 1 → Redeem xSOL → Mode 2 → Fee: 4 → 8
- Normal → Mint stable → Mode 1 → Fee: 0.1 → 0.5
- Normal → Swap to stablecoin → Mode 1 → Fee: 1 → 4

However, in other cases it causes **unintended overcharging**:

- Mode 1 → Mint xSOL → Normal → Fee: 0.5 → 1
- Mode 1 → Redeem stable → Normal → Fee: 0.1 → 0.3
- Mode 1 → Swap from stablecoin → Normal → Fee: 0.5 → 1

In these cases, the transaction **improves protocol stability** (moving toward safer mode), yet users are penalized with higher fees than they should pay.

Location

https://github.com/hylo-so/protocol/blob/2381c3602dae7bc0be1e2576d2824dd69fab2ba9/programs/hylo-exchange/src/exchange_context.rs#L234-L236 . . .

Mitigation Suggestion

For transactions that move the system to a **more stable mode**, fees should be calculated based on the **current mode**, not the projected one.

Remediation

Fixed in commit [1793c40c52cd5a159a5a0c6168012be7a799801e](#).

Description

We found that the **swap instruction** depends on several invariants to calculate fair exchange rates:

- Current oracle prices (`sol_usd_pyth_feed`)
- Current token supplies (`levercoin_mint.supply`)
- Current protocol state (e.g., total SOL, collateral ratio)

This makes it **vulnerable to frontrunning attacks**. Currently, there is **no slippage protection mechanism** to guard against this.

Location

<https://github.com/hylo-so/protocol/blob/main/programs/hylo-exchange/src/instructions/swap.rs>

Mitigation Suggestion

Add a **slippage check** to the swap instruction so that the user specifies a minimum acceptable output. If the actual result falls below this bound, the transaction should revert.

Remediation

Fixed in commit `f70f4643d4c44949e4d6b314e30989d812f7d97a`.

Description

We found that **harvest yield is calculated using the current LST balance**, which creates inconsistencies when deposits or withdrawals occur **before** the harvest is executed.

Example Walkthrough

- **Epoch A**
- Price = 10
- `last_yield_harvest_epoch = A`
- Total LST = 100
- **Epoch B (Price update)**
- Price = 11
- `prev_price = 10`

Now, yield harvest should calculate:

$$(11 - 10) * 100$$

Problem Scenario 1: Deposit before harvest

- A user deposits 10 LST before calling harvest.
- New total = 110 LST.
- Yield calculation becomes:

$$(11 - 10) * 110 = 110 \text{ yield}$$

The extra 10 yield is incorrect, since the new deposit did not generate yield during the previous epoch.

Problem Scenario 2: Withdrawal before harvest

- A user withdraws 10 LST before calling harvest.
- New total = 90 LST.
- Yield calculation becomes:

$$(11 - 10) * 90 = 90 \text{ yield}$$

This undercounts yield, because those 10 withdrawn LST actually did contribute yield during the previous epoch.

Impact

- Depositors can **steal yield** by depositing right before harvest.
- Withdrawers can **burn yield** by withdrawing right before harvest.
- This causes unfair yield distribution and accounting mismatches.

Location

https://github.com/hylo-so/protocol/blob/2381c3602dae7bc0be1e2576d2824dd69fab2ba9/programs/hylo-exchange/src/state/lst_registry.rs#L169-L176

Relevant Code

```
let appreciation = lst.lst_header.sol_price_appreciation()?;
let amount_lst = UFix64::<N9>::new(lst.vault.amount);
let amount_sol = appreciation
    .mul_div_floor(amount_lst, UFix64::one())
    .ok_or(LstSolAppreciation)?;
acc
    .checked_add(&amount_sol)
    .ok_or(LstAdditionOverflow.into())
```

Mitigation Suggestion

- Calculate yield based on the **LST supply at the last `prev_price` epoch**, not the current LST balance.
- This ensures only the tokens present during the entire epoch accrue yield.

Note: After this fix, if yield harvest is skipped for an epoch, LST supply changes in that skipped epoch may not be fully accounted for. This introduces a new consistency risk that must be handled (e.g., by enforcing harvest at the first of epoch or before deposits/withdrawals, or snapshotting balances per epoch).

Remediation

Final fix applied in commit [092858727691b386f80e7a1ccec5931b6a060195](#). The final fix locks the use of mint/redeem/swam in a new epoch until the rewards are harvested, enforcing correct order of operations.

Description

We found that when **updating LST prices**, the program also updates `prev_price_sol`. Since `prev_price_sol` is later used to calculate yield, this creates a problem:

If yield harvest is **not called during the epoch when the price changes**, the unclaimed yield from that epoch is lost.

Example Walkthrough

• Epoch A

- Prev Price = 10

- Price = 10

- `last_yield_harvest_epoch = A`

• Epoch B

- Price updates → 11

- `prev_price_sol` updates → 10

- If `harvest_yield` is not called in this epoch, the yield between 10 → 11 is lost.

• Epoch C

- Price updates → 12

- `prev_price_sol` updates → 11

- When `harvest_yield` is finally called, it only calculates **12 - 11** instead of the correct **12 - 10**.

Impact

This results in **under-counting yields** and directly causes yield loss if harvest is skipped for an epoch.

Proof of Concept

```
#[test]
fn lst_header_update_next_epoch() -> anyhow::Result<()> {
    let price_one = LstSolPrice {
        price: UFix64::<N9>::new(42).into(),
        epoch: 0,
    };
    let price_two = LstSolPrice {
        price: UFix64::<N9>::new(69).into(),
        epoch: 0,
    };
    let mut header = make_header(price_one, price_two);

    let next_price = UFix64::new(420);
    // Change price but in new epoch
    header.update_price(next_price, 1)?;
    let one = header.sol_price_appreciation()?;

    let next_price_2 = UFix64::new(430);
    // Change price but in new epoch
    header.update_price(next_price_2, 2)?;
    let two = header.sol_price_appreciation()?;

    // Yield from epoch 1 is lost, so appreciation is smaller in epoch 2
    assert!(one > two); // @audit

    Ok(())
}
```

```
}
```

As shown, the **appreciation calculated after the second update is smaller**, because the unclaimed yield from the first update is never included.

Location

https://github.com/hylo-so/protocol/blob/317018e3c48c9d87598f1cd0c45aad005826aec/programs/hylo-exchange/src/state/lst_registry.rs#L277-L301

Relevant Code

```
pub fn update_price(
    &mut self,
    new_price: UFix64<N9>,
    this_epoch: u64,
) -> Result<Self> {
    match this_epoch.cmp(&self.price_sol.epoch) {
        Ordering::Greater => {
            // Epoch advanced, store previous price and update
            self.prev_price_sol = self.price_sol;
            self.price_sol = LstSolPrice::new(new_price.into(), this_epoch);
            Ok(*self)
        }
        Ordering::Equal => {
            // Same epoch, in-place price update
            self.price_sol = LstSolPrice::new(new_price.into(), this_epoch);
            Ok(*self)
        }
        Ordering::Less => Err(LstPriceEpochsInvalid.into()),
    }
}

/// Marginal price change for this LST from the previous epoch to now.
pub fn sol_price_appreciation(&self) -> Result<UFix64<N9>> {
    self.price_sol.checked_delta(&self.prev_price_sol)
}
```

Mitigation Suggestion

Only update **prev_price_sol** when yield is actually harvested, update it in **harvest_yield**. This ensures that **all skipped epochs' appreciation is preserved** and properly accounted for at the next harvest.

Remediation

Marked as wontfix, because the behavior of only allowing marginal epoch yield to be extracted is intentional.

ACC-L1 Admin can reinitialize lst registry

LOW

FIXED

Description

We found that the protocol admin may call the `InitializeLstRegistry` multiple times, and in particular a second time after the protocol has been already initialized. This would overwrite an existing `lst_registry` with an empty look up table, and potentially brick the contract, as LSTs can't be re-registered, because their `lst_header` account already exists.

Location

https://github.com/hylo-so/protocol/blob/2381c3602dae7bc0be1e2576d2824dd69fab2ba9/programs/hylo-exchange/src/instructions/initialize_lst_registry.rs#L7-L34 <https://github.com/hylo-so/protocol/blob/2381c3602dae7bc0be1e2576d2824dd69fab2ba9/programs/hylo-exchange/src/state/hylo.rs#L85-L93>

Relevant Code

```
/// initialize_lst_registry.rs L7-L34
#[derive(Accounts)]
pub struct InitializeLstRegistry<'info> {
    #[account(mut)]
    pub admin: Signer<'info>,

    #[account(
        mut,
        has_one = admin,
        seeds = [HYLO],
        bump,
    )]
    pub hylo: Account<'info, Hylo>,

    /// CHECK: PDA
    #[account(
        seeds = [LST_REGISTRY_AUTH],
        bump,
    )]
    pub registry_auth: UncheckedAccount<'info>,

    /// CHECK: New account for registry based on slot
    #[account(mut)]
    pub lst_registry: AccountInfo<'info>,

    pub lut_program: Program<'info, AddressLookupTable>,

    pub system_program: Program<'info, System>,
}

/// hylo.rs L85-L93
pub fn init_lst_registry(
    &mut self,
    lst_registry: Pubkey,
    registry_auth_bump: u8,
) -> Result<()> {
    self.lst_registry = lst_registry;
    self.registry_auth_bump = registry_auth_bump;
    Ok(())
}
```

Mitigation Suggestion

Add a check that when initializing the LST registry, no prior LST registry exists yet, so that this becomes a one-shot instruction.

Remediation

Fixed in commit [9e3426898d839a3d897b599f24e12b3f46f24e3a](#).

Description

We found that the protocol incorrectly calculates `projected_stability_mode` during:

- Mint Stablecoin
- Mint Levercoin
- Stable → Lever swap

Currently, the calculation assumes the **full** `amount_lst` is deposited into the protocol. However, in reality only `amount_lst - fee` is deposited.

Example

- Current mode: **1**
- Threshold: deposit $\geq$ 100 LST → move to **Normal** mode
- User deposits **100 LST**
- `projected_stability_mode` is set to **Normal** (based on 100 LST)
- But only **100 - fee** enters the protocol → actual mode remains **1**

This creates a **mismatch** between projected and actual stability mode, which could make users pay too many fees, prevent legitimate interactions, or allow illegal state changes.

Location

https://github.com/hylo-so/protocol/blob/2381c3602dae7bc0be1e2576d2824dd69fab2ba9/programs/hylo-exchange/src/instructions/mint_levercoin.rs#L146-L153

https://github.com/hylo-so/protocol/blob/2381c3602dae7bc0be1e2576d2824dd69fab2ba9/programs/hylo-exchange/src/instructions/mint_levercoin.rs#L167-L175

Relevant Code

```
// Compute fees from LST deposit
let FeeExtract {
    fees_extracted,
    amount_remaining,
} = exchange_context.levercoin_mint_fee(
    &self.lst_header.price_sol,
    UFix64::new(amount_lst_to_deposit),
)?;

// Update TotalSolCache with user's deposit in SOL
let sol_deposited = self
    .lst_header
    .price_sol
    .convert_sol(amount_remaining, exchange_context.clock.epoch())?;
self
    .hylo
    .total_sol_cache
    .increment(sol_deposited, exchange_context.clock.epoch())?;
```

Mitigation Suggestion

Calculate `projected_stability_mode` based on the **net amount deposited or withdrawn** (`amount_lst - fee`), not the gross amount.

Remediation

This issue has been closed as won't fix. It may be addressed in v2.

Description

The **Stability Pool** is vulnerable to a **first share manipulation attack**, similar to [TOB-YEARN-003](https://github.com/yearn/yearn-security/blob/master/audits/20210719_ToB_yearn_vaultsv2/ToB_-_Yearn_Vault_v_2_Smart_Contracts_Audit_Report.pdf).

In this attack, the very first depositor (or a malicious actor) can manipulate the pool's NAV by donating tokens, which artificially inflates the share price:

Attack flow:

1. LP token NAV starts at 1. 2. Attacker deposits 1 token → receives 1 share. 3. Attacker donates additional tokens directly into the pool (without minting shares). 4. Pool NAV per share increases significantly. 5. other users' deposits (get a smaller share) 6. Attacker withdraws, redeeming shares for more than their fair value. This results in unfair dilution of future depositors who join after the manipulation.

Currently, since the protocol is live and has a **large number of LP tokens in circulation**, the attack surface is reduced. However, the vulnerability still exists in edge cases (e.g., a new pool deployment, low-liquidity scenarios, or testing environments).

Location

https://github.com/hylo-so/protocol/blob/2381c3602dae7bc0be1e2576d2824dd69fab2ba9/programs/hylo-stability-pool/src/instructions/user_deposit.rs#L119

Mitigation Suggestion

Adopt the **Uniswap V2 approach** to prevent first-share manipulation:

- When `totalSupply() == 0`, mint a minimum number of LP tokens (e.g., 1000) to the address that is not controlled by anyone (PDA).
- This ensures that the first depositor cannot artificially inflate share value and extract unfair profits.

Remediation

This issue won't be fixed as the protocol has already been deployed and for the current deployment the attack doesn't apply.

Description

In the **Stability Pool** implementation, the **UpdateAdmin** instruction currently requires the **admin** account to be a **signer**.

This is inconsistent with the **Exchange UpdateAdmin**, which does not enforce the same restriction.

The problem is that requiring the **current admin's signature** makes it impossible to rotate the admin if:

- The admin's key is lost.
- The admin's key is compromised.
- The admin is unresponsive.

In such cases, the upgrade authority should still be able to update the admin, but the current implementation blocks it.

Location

https://github.com/hylo-so/protocol/blob/2381c3602dae7bc0be1e2576d2824dd69fab2ba9/programs/hylo-stability-pool/src/instructions/update_admin.rs#L10-L14

Relevant Code

```
pub struct UpdateAdmin<'info> {  
    #[account(mut)]  
    pub admin: Signer<'info>,  
  
    pub upgrade_authority: Signer<'info>,
```

Mitigation Suggestion

Remove the **Signer** requirement for **admin**. Instead, rely only on the **upgrade_authority** to authorize admin rotation.

Remediation

Fixed in commit [63d16b9782016236c4c0194c18d3f2829a8afc73](#).

Description

In **depeg mode**, when the stablecoin NAV falls below **1.0**, the protocol should set the **levercoin NAV** to **0**. However, the current implementation still produces a small non-zero value instead of zero.

Example from the test:

```
#[test]
fn depeg_stablecoin_leverage_not_zero() -> Result<()> {
    let total_sol = UFix64::<N9>::new(1_000_000_000);
    let usd_sol_price = UFix64::<N8>::new(20_000_000_000);
    let amount_stablecoin = UFix64::<N6>::new(210_000_000);
    let lever_supply = UFix64::<N6>::new(1_000_000);

    let c_ra = collateral_ratio(total_sol, usd_sol_price, amount_stablecoin)?;
    assert!(c_ra < UFix64::one());

    let nav = depeg_stablecoin_nav(total_sol, usd_sol_price, amount_stablecoin)?;
    assert!(nav < UFix64::one());

    let nav2 = next_levercoin_nav(
        total_sol,
        usd_sol_price,
        amount_stablecoin,
        nav,
        lever_supply,
    ).unwrap();

    assert_eq!(UFix64::zero(), nav2);
    Ok(())
}
```

But the actual output is non-zero:

```
thread 'nav::tests::depeg_stablecoin_leverage_not_zero' panicked at programs/hylo-exchange/src/nav.rs:240:5:
assertion `left == right` failed
  left: 0x10^-6
 right: 200x10^-6
```

This means **levercoin NAV** is **incorrectly retained with a small positive value** instead of being set to zero.

Location

<https://github.com/hylo-so/protocol/blob/2381c3602dae7bc0be1e2576d2824dd69fab2ba9/programs/hylo-exchange/src/nav.rs#L97>

Mitigation Suggestion

Update the `next_levercoin_nav` calculation to explicitly check for **depeg conditions** (`stablecoin_nav < 1.0`) and **force levercoin NAV = 0** in those cases.

Remediation

Fixed in commit [1456ce576cd8052436a7e7e63da2c0d18a2cb238](#).

Description

We found that in multiple instructions, the protocol **forces users to use their Associated Token Accounts (ATA)** for stablecoin, levercoin, and LST tokens.

This restriction is unnecessary and reduces flexibility for users and potential integrations. For example, users may want to deposit or withdraw using a custom token account they control, rather than being forced to interact only through their ATA.

By enforcing ATA usage, the protocol limits user choice without providing additional safety, since ownership and mint checks can be performed directly on any token account.

Location

https://github.com/hylo-so/protocol/blob/2381c3602dae7bc0be1e2576d2824dd69fab2ba9/programs/hylo-exchange/src/instructions/mint_levercoin.rs#L76-L88 <https://github.com/hylo-so/protocol/blob/2381c3602dae7bc0be1e2576d2824dd69fab2ba9/programs/hylo-exchange/src/instructions/swap.rs#L64-L69> . . .

Relevant Code

```
#[account(
    mut,
    associated_token::mint = lst_mint,
    associated_token::authority = user,
)]
pub user_lst_ata: Box<Account<'info, TokenAccount>>,

#[account(
    mut,
    associated_token::authority = user,
    associated_token::mint = levercoin_mint,
)]
pub user_levercoin_ata: Account<'info, TokenAccount>,
```

Mitigation Suggestion

Allow users to provide **any valid token account** they control, rather than forcing the use of ATA accounts.

Remediation

Fixed in commit [b696b023605cb0967e0b28020446ccdb52aa4668](#).

ACC-L7

Incorrect Validation of Harvestable Stablecoin Amount in `yield\_harvest`

LOW

FIXED

Description

We found that `yield_harvest` validates the **maximum mintable amount** against `stablecoin_amount`. This is incorrect because the protocol only mints the **fee portion** and the **allocated amount to the pool**, not the full `stablecoin_amount`.

Because of this mismatch, the validation may incorrectly reject valid harvest operations.

Location

https://github.com/hylo-so/protocol/blob/2381c3602dae7bc0be1e2576d2824dd69fab2ba9/programs/hylo-exchange/src/instructions/harvest_yield.rs#L146

Relevant Code

```
exchange_context.validate_stablecoin_amount(stablecoin_amount)?
```

Mitigation Suggestion

Update the validation to check against the **fee + allocated stablecoin amounts** instead of the raw `stablecoin_amount`.

Remediation

Fixed in commit [283b3b5da465c8e8907e71e7ffb4500025fb43dc](#).

Description

We found that `harvest_yield` directly mints stablecoins, which changes the **collateral ratio**. Since collateral ratio is used to determine minting and redemption fees, the timing of `harvest_yield` calls can significantly affect user costs and even disable minting.

Example

- `harvest_yield` mints 100 stablecoins.
 - **Case 1: Without harvest before mint**
 - User mints stablecoins.
 - Collateral ratio is calculated on old state (before yield).
 - Fee is based on outdated, high collateral ratio.
 - **Case 2: With harvest before mint**
 - Collateral ratio is updated (lower due to new stable supply).
 - Minting fee increases, or in some cases minting may even be disabled.
- This creates inconsistent and unfair user experiences depending on whether `harvest_yield` is called first.

Impact

- Users may pay different fees for the same action depending on timing.
- In extreme cases, minting may be blocked even though it would have been allowed just before yield harvesting.

Location

https://github.com/hylo-so/protocol/blob/2381c3602dae7bc0be1e2576d2824dd69fab2ba9/programs/hylo-exchange/src/exchange_context.rs#L160-L187

Relevant Code

```
pub fn stablecoin_mint_fee(
    &self,
    lst_sol_price: &LstSolPrice,
    amount_lst: UFix64<N9>,
) -> Result<FeeExtract<N9>> {
    // Total SOL being added
    let new_sol = lst_sol_price.convert_sol(amount_lst, self.clock.epoch());
    let new_total_sol = self
        .total_sol
        .checked_add(&new_sol)
        .ok_or(DestinationFeeSol)?;

    // Total stablecoin after mint
    let new_total_stablecoin = self
        .token_conversion(lst_sol_price)?
        .lst_to_token(amount_lst, self.stablecoin_nav())?
        .checked_add(&self.stablecoin_supply)
        .ok_or(DestinationFeeStablecoin)?;

    // Base fee calculation on projected stability mode
    let projected_stability_mode =
```

```
self.projected_stability_mode(new_total_sol, new_total_stablecoin)?;  
  
self  
  .stablecoin_fees  
  .mint_fee(projected_stability_mode)  
  .and_then(|fee| FeeExtract::new(fee, amount_lst))  
}
```

Mitigation Suggestion

Enforce that **harvest_yield** is called at the **start of each epoch** before allowing minting or redemption. This ensures the collateral ratio is always up to date and consistent for all users. And we must share that **harvest_yield** never revert and always return `Ok()`.

Remediation

Fixed together with <https://github.com/accretion-xyz/2025-hylo-hylo-audit-A25HYL1/issues/7>.

Description

We found that no checks on the fee percentage are made when updating them with `update_stablecoin_fees` or `update_levercoin_fees`. Further, no checks are made in the initial program initialization. This can lead to an accidental DoS of minting, redeeming, and swapping of levercoins and stablecoins. Given the modularity of fees, not all functions may be affected at the same time. When fees above 100% are set, `checked_sub` in `FeeExtract::new` will return an error that will cause the instruction to fail because `fees_extracted` will be higher than `amount_in`.

This bug also exists inside the HyloStability program and is affecting the withdrawal fees.

Location

https://github.com/hylo-so/protocol/blob/317018e3c48c9d87598f1cd0c45aad005826aec/programs/hylo-exchange/src/state/fee_controller.rs#L42-L60 https://github.com/hylo-so/protocol/blob/317018e3c48c9d87598f1cd0c45aad005826aec/programs/hylo-stability-pool/src/state/pool_config.rs#L47-L57

Relevant Code

```
impl<Exp> FeeExtract<Exp> {
    pub fn new(
        fee: UFix64<N4>,
        amount_in: UFix64<Exp>,
    ) -> Result<FeeExtract<Exp>> {
        let fees_extracted = amount_in
            .mul_div_ceil(fee, UFix64::::one())
            .ok_or(FeeExtraction)?;

        let amount_remaining = amount_in
            .checked_sub(&fees_extracted)
            .ok_or(FeeExtraction)?;

        Ok(FeeExtract {
            fees_extracted,
            amount_remaining,
        })
    }
}
```

Mitigation Suggestion

Adding a validation method to `FeePair` and calling it in the update and initialization instructions will prevent the scenario from happening before the update goes through:

```
impl FeePair {
    pub fn assert_valid(&self) {
        let Ok(mint) = self.mint() else {
            panic!("Exit here - We can also return a Result::Err")
        };
        let Ok(redeem) = self.redeem() else {
            panic!("Exit here - We can also return a Result::Err")
        };

        if !(mint < UFix64::::one() && redeem < UFix64::::one()) {
            panic!("Exit here - We can also return a Result::Err")
        }
    }
}
```

In the Hylo Stability pool, in the `update_withdrawal_fee` function, we can add the following:

```
/// pool_config.rs L47-L57
pub fn update_withdrawal_fee(
    &mut self,
    new_withdrawal_fee: UFixValue64,
) -> Result<UpdateWithdrawalFeeEvent> {
    let old_withdrawal_fee = self.withdrawal_fee;
    self.withdrawal_fee = new_withdrawal_fee;
    require!(
        self.withdrawal_fee()? < UFix64::<N4>::one(),
        ErrorCode::FeeExtraction,
    );
    Ok(UpdateWithdrawalFeeEvent {
        old_withdrawal_fee,
        new_withdrawal_fee,
    })
}
```

A similar validation exists in the [YieldHarvestConfig](#)

Remediation

Remediated in <https://github.com/hylo-so/protocol/commit/5ce6b8c534e3dad4bba0dcadc3b08b17e32e8276>

Description

We found that the protocol is vulnerable to a LST portfolio rebalancing attack if the protocol allows multiple different LSTs to be deposited. If issuance of one of those LSTs is advantageous to a user (e.g. the validator who receives this LST's stake), they might intentionally rebalance the protocol's LST holdings. Let's say the protocol supports three LSTs, jitoSOL, hyloSOL, and bobSOL. Bob runs the bobSOL validator.

Protocol Holds: 50% jitoSOL 45% hyloSOL 5% bobSOL

Bob now may do the following: 1. Deposit N bobSOL, receive hyUSD 2. redeem hyUSD for jitoSOL or hyloSOL 3. redeem jitoSOL/hyloSOL and mint bobSOL, increasing their own validator stake 4. Deposit bobSOL, receive hyUSD Loop

Until the protocol will hold 0% jitoSOL 0% hyloSOL 100% bobSOL

This is possible because the protocol does not enforce any LST balance proportions. As the protocol currently only supports jitoSOL, this isn't an issue, but it will become an issue as LSTs are added.

It is worth noting that the Hylo team controls which LSTs are allowed within the protocol, and they plan to only allow big ecosystem LSTs which should prevent this issue by incentives.

Location

https://github.com/hylo-so/protocol/blob/2381c3602dae7bc0be1e2576d2824dd69fab2ba9/programs/hylo-exchange/src/state/lst_registry.rs#L42-L42

Relevant Code

```
/// lst_registry.rs L42-L42
pub struct LstRegistry<'info> {
```

Mitigation Suggestion

When adding new LSTs to the system, you should avoid single-validator LSTs, and potentially enforce an LST portfolio balance mechanism.

Remediation

Work in progress. Currently, only jitoSol is supported which prevents this issue. The team plans to only add LSTs which are incentivized to participate honestly.

Description

The current comments and implementation of the stability pool indicate that rebalancing should be allowed in depeg mode. However, in the Hylo exchange swap instruction, swapping from stable to leverage is only permitted starting from `StabilityMode::Mode2`.

Location

<https://github.com/hylo-so/protocol/blob/2381c3602dae7bc0be1e2576d2824dd69fab2ba9/programs/hylo-stability-pool/src/instructions/rebalance.rs#L139-L144>

Mitigation Suggestion

Allow the rebalance mechanism only in `StabilityMode::Mode2`.

Remediation

Fixed in commit `81c7fa55eb5b7b6d1e4b80fb11206d4bc1691891`.

APPENDIX

Vulnerability Classification

We rate our issues according to the following scale. Informational issues are reported informally to the developers and are not included in this report.

Severity	Description
Critical	Vulnerabilities that can be easily exploited and result in loss of user funds, or directly violate the protocol's integrity. Immediate action is required.
High	Vulnerabilities that can lead to loss of user funds under non-trivial preconditions, loss of fees, or permanent denial of service that requires a program upgrade. These issues require attention and should be resolved in the short term.
Medium	Vulnerabilities that may be more difficult to exploit but could still lead to some compromise of the system's functionality. For example, partial denial of service attacks, or such attacks that do not require a program upgrade to resolve, but may require manual intervention. These issues should be addressed as part of the normal development cycle.
Low	Vulnerabilities that have a minimal impact on the system's operations and can be fixed over time. These issues may include inconsistencies in state, or require such high capital investments that they are not exploitable profitably.
Informational	Findings that do not pose an immediate risk but could affect the system's efficiency, maintainability, or best practices.

Audit Methodology

Accretion is a boutique security auditor specializing in Solana's ecosystem. We employ a customized approach for each client, strategically allocating our resources to maximize code review effectiveness. Our auditors dedicate substantial time to developing a comprehensive understanding of each program under review, examining design decisions, expected and edge-case behaviors, invariants, optimizations, and data structures, while meticulously verifying mathematical correctness—all within the context of the developers' intentions.

Our audit scope extends beyond on-chain components to include associated infrastructure, such as user interfaces and supporting systems. Every audit encompasses both a holistic protocol design review and detailed line-by-line code analysis.

During our assessment, we focus on identifying:

- Solana-specific vulnerabilities
- Access control issues
- Arithmetic errors and precision loss
- Race conditions and MEV opportunities
- Logic errors and edge cases
- Performance optimization opportunities
- Invariant violations
- Account confusion vulnerabilities
- Authority check omissions
- Token22 implementation risks and SPL-related pitfalls
- Deviations from best practices

Our approach transcends conventional vulnerability classifications. We continuously conduct ecosystem-wide security research to identify and mitigate emerging threat vectors, ensuring our audits remain at the forefront of Solana security practices.